Image Processing - Exercise 3

Shay Morad

Introduction

The goal of this exercise is to explore and implement image blending techniques to create seamless and visually compelling combinations of different images. The primary focus is on two key tasks: blending two images seamlessly using a mask and generating hybrid images that display different appearances based on the viewer's distance. These tasks leverage foundational concepts in image processing, such as frequency domain transformations, Gaussian filtering, and pyramidal decomposition, to manipulate and combine visual data creatively. The main technique I've used in this exercise, in order to solve it, was pyramidal decomposition and blending.

Algorithm

Description:

In order to solve Task 1.1 Image Blending, these are the steps involved in my algorithm:
1. Load the images (2 images to blend and a mask).
2. Construct the Gaussian pyramids for each image and calculate the Laplacian pyramids for the 2 images meant to be blended (meaning not for the mask image).
3. Construct a new Laplacian pyramid by blending the former Laplacian pyramids of the images using the Gaussian pyramid of the mask.
4. Reconstruct the blended image from the blended Laplacian pyramid.

The algorithm followed what was mainly showed in Lecture 6, Week 7 of the semester by Shmuel Peleg.

In order to solver Task 1.2 Hybrid Images, these are the steps involved in my algorithm:
1. Load the images.
2. Calculate the FT of each image.
3. Create a Gaussian filter in the Fourier domain.
4. Apply the Gaussian filter in the Fourier domain to one of the images, and apply (1-Gaussian filter) to the other image.
5. Combine the two Fourier filtered Fourier transforms.
6. Calculate IFT on the combined result, getting us the new hybrid image.

The algorithm followed what was mainly showed in Recitation 7, Week 7 of the semester by Leeyam Gabay.

Implementation Details:

Both tasks were implemented using the existing libraries; NumPy and OpenCV.

Task 1.1 Image Blending was implemented as follows – I loaded the images and the mask using 'cv2.imread' function. To make sure the mask is binary, I divided it by 255 as it's the highest value for pixel intensity we want and rounded it, and as such closer values to black are 0 and closer values to white are 1. Afterwards, I resized the images and the mask to a constant NxN size which can be changed (hyper-parameter), to make sure everything has the same dimensions to avoid conflicts and issues in the future of the algorithm. Then, I calculated the pyramid levels and took the minimum between the calculated maximum possible level and a constant (hyper-parameter). Then the Gaussian pyramids of all were calculated using the 'cv.pyrDown' function, of which the images Gaussian pyramids were used to calculate their Laplacian pyramids as shown in the lecture – expanding the higher level, resizing it, and substracting it from the current level, and doing that for all levels. In order to expand I used the 'cv.pyrUp' function, and then resized it using the 'cv.resize' function. After calculating the mask's Gaussian pyramid and the images Laplacian pyramids, I blended them using the mask. I did that by multiplying the first image L-pyramid by the mask's G-pyramid and adding to that the second image's L-pyramid multipied by (1 – G-pyramid) of the mask. To end it all, the new image was reconstructed by summing all the blended L-pyramid levels from top to bottom, by expanding and resizing each level using 'cv.pyrUp' and 'cv.resize'.
The 2 hyper parameters were used for conveniency and can be changed. The reason for using a parameter for the pyramid levels is because sometimes, using too many levels (for example 9 instead of 5) can result in a brighter blend.

Task 1.2 Hybrid Images was implemented as follows – I loaded the images using 'cv.imread' but with a flag specifying to load them in grayscale. Afterwards the images were resized to size NxN for similar reasons mentioned before. Then the FT of the images was calculated using numpy's function 'np.fft.ff2'. The FT of the images were shifted to center the by the zero frequency using the function of numpy 'np.fft.fftshift'. Then, a Gaussian filter was made by using the formula shown in lectures 3&4. The first Image FT was multiplied by the Gaussian filter, and the second Image FT was multipled by (1-Gaussian filter), as Implied to do so in the recitation. Then the two FT's were added together. In order to get the hybrid image, the inverse FT was calculated by using numpy's IFT function – 'np.fft.ifft2'. The real part was extracted as well using numpy's function 'np.real' to make sure we don't have complex leftovers. To make sure the hybrid image is in correct intensity values and data type, it was normalized to be between the intensities [0,255]. The main challenge I had in this task is the Gaussian filter. At first I tried using OpenCV's gaussian filter, but for many reasons the final product just did not turn out well, and in order to overcome this, I used Google and past Lectures information to create directly a Gaussian filter. A hyper-parameter used here was the Gaussian's "spread", and as using a low one (32 in my implementation, can be changed), it means we take the low frequencies from the first image and the higher ones from the second one, resulting in the wanted hybrid image where the first one

looks blurry and clear from a far distance and the second one more visible from a closer look.
The core difference between the algorithms is that the first algorithm works in the Real (spatial) domain, while the second one works in the Fourier domain.

Results

For image blending, these are the images I used for blending, the mask, and the result:

Image 1



Image 2



mask



blended image

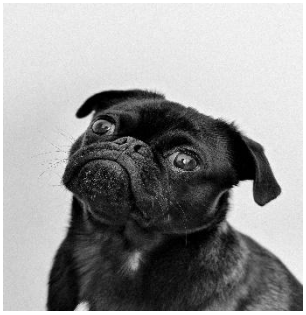


For the hybrid image, these are the images used, gaussian filter and result:

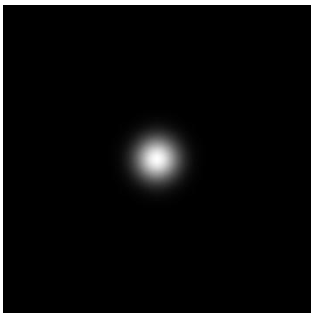
Image 1



Image 2



low-pass gaussian filter



hybrid image

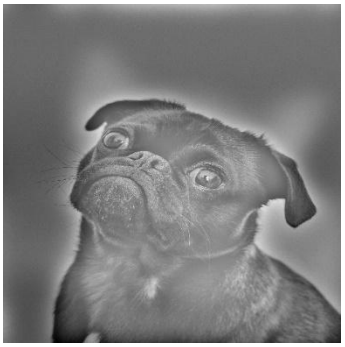


Examples for failed results:

failed blending



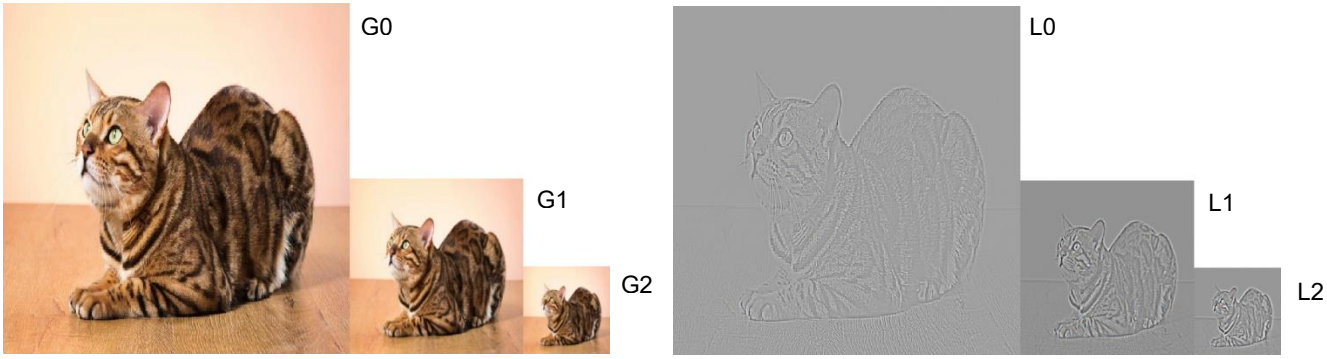
failed hybrid



As we can see, the blending failed in the fact that there is not really a blend. The colors mismatch between the edge of the first image to the edge of the second image, and in general not just the edges, the face of the lion looks much brighter than the cat's body, and it looks like we just copy-pasted the lion's face on the cat's face without really blending. The reason for that, I suspect, is the usage of too few pyramid levels.
As for the hybrid image, it failed in that we get no information at all from the cute cat image. The reason for that, I suspect, is the usage of a too small spread parameter for the Gaussian filter, and as such, what happens is we take almost all the information from one image and not enough information from the other one.

Pyramids

First 3 levels of the Gaussian and Laplacian pyramids of image 1 from the image blending:



As for the Gaussian pyramid, each level is a progressively blurred and down-sampled version of the original image. The higher levels (smaller images) contain more coarse details of the image. The resolution decreases with each level.
For the Laplacian pyramid, each level represents the difference between adjacent levels of the Gaussian pyramid. The levels highlight edges and details at different scales.

Conclusion

In this exercise, we explored image blending techniques using Gaussian and Laplacian pyramids, as well as the creation of hybrid images through Fourier domain manipulation. By applying these methods, we successfully achieved seamless transitions between images and observed the interplay of spatial and frequency domains in image processing. The Gaussian and Laplacian pyramids facilitated gradual blending, while the hybrid image technique highlighted the capability to combine low and high-frequency details into a single image. These experiments not only demonstrated the practical applications of the learned algorithms but also showcased the creative potential of image manipulation in both domains.